Report Automation

This project showcases a VBA script I developed to automate a complex Excel reporting workflow. The script performs a multi-step process that includes:

- Deleting unnecessary “filler” columns and rows
- Applying a counselor filter and deleting specific rows based on filter criteria, then clearing filters
- Applying a state filter and deleting specific rows based on filter criteria, then clearing filters
- Adding a custom “Notes” column
- Writing a robust VLOOKUP formula with built-in logic to handle edge cases and variations in report size
- Converting the VLOOKUP-generated notes to static values, ensuring the report remains stable and editable without breaking formulas
- Sorting the notes column in desired fashion to make the report ready for use and analysis

This process used to take approximately 2 minutes when performed manually. The final macro now completes all actions automatically in about **9 seconds** with a single script execution.

VBA Script

```
function main(workbook: ExcelScript.Workbook) {


  let selectedSheet = workbook.getActiveWorksheet();




  // Delete range 1:4 on selectedSheet "filler rows"
```

```
selectedSheet.getRange("1:4").delete(ExcelScript.DeleteShiftDirection.up);

// Delete range A:F on selectedSheet "junk columns"

selectedSheet.getRange("A:F").delete(ExcelScript.DeleteShiftDirection.left);

// Delete range B:D on selectedSheet "junk columns"

selectedSheet.getRange("B:D").delete(ExcelScript.DeleteShiftDirection.left);

// Updated: Only delete columns D:G, leave H and I

selectedSheet.getRange("D:G").delete(ExcelScript.DeleteShiftDirection.left);



// Get the used range after deletions

let usedRange = selectedSheet.getUsedRange();

if (!usedRange) {

  console.log("Sheet is empty.");
```

```
  return;

}


let values = usedRange.getValues();

const headerRow = values[0] as string[];


// Dynamically find the "FC" column index

const fcColIndex = headerRow.indexOf("FC");

if (fcColIndex === -1) {

  console.log(`"FC" column not found. Aborting.`);

  return;

}
```

```
// Collect the row indexes to delete (skip header row 0) "filter out and delete SSC's"

let rowsToDelete: number[] = [];

for (let i = 1; i < values.length; i++) {

  let cellValue = values[i][fcColIndex];

  if (cellValue === "Counselor X" || cellValue === "Counselor Y") {

    rowsToDelete.push(i);

  }

}



// Delete rows from bottom up to avoid messing up indexes

for (let i = rowsToDelete.length - 1; i >= 0; i--) {
```

```
    selectedSheet.getRange((rowsToDelete[i] + 1) + ":" + (rowsToDelete[i] +
1)).delete(ExcelScript.DeleteShiftDirection.up);

  }



  // Optionally, clear filters if applied

  let autoFilter = selectedSheet.getAutoFilter();

  if (autoFilter) {

    autoFilter.clearCriteria();

  }



  // STEP 3: Delete rows where State == "OH"

  let usedRangeStep3 = selectedSheet.getUsedRange();

  if (!usedRangeStep3) return;
```

```
let valuesStep3 = usedRangeStep3.getValues() as string[][];



// Find "State" column index

const headerRowStep3 = valuesStep3[0];

const stateColIndex = headerRowStep3.indexOf("State");

if (stateColIndex === -1) {

  console.log(`Could not find "State" column.`);

  return;

}



// Track rows to delete

let rowsToDeleteStep3: number[] = [];
```

```
  for (let i = 1; i < valuesStep3.length; i++) {

    if (valuesStep3[i][stateColIndex] === "OH") {

      rowsToDeleteStep3.push(i);

    }

  }




  // Delete rows from bottom up

  for (let i = rowsToDeleteStep3.length - 1; i >= 0; i--) {

    selectedSheet.getRange(`${rowsToDeleteStep3[i] + 1}:${rowsToDeleteStep3[i] +
1}`).delete(ExcelScript.DeleteShiftDirection.up);

  }




  // STEP 4: Insert "Notes" column after "Stu Num" and freeze top row
```

```
let finalUsedRange = selectedSheet.getUsedRange();

if (!finalUsedRange) return;

let finalValues = finalUsedRange.getValues() as string[][];

let finalHeaders = finalValues[0];



let stuNumIndex = finalHeaders.indexOf("Stu Num");

if (stuNumIndex === -1) {

  console.log(`"Stu Num" column not found.`);

  return;

}



let insertColIndex = stuNumIndex + 1;
```

```
  let colLetter = getColumnLetter(insertColIndex + 1); // Excel is 1-based




  selectedSheet.getRange(`${colLetter}:${colLetter}`).insert(ExcelScript.InsertShiftDirection.
right);

  selectedSheet.getRange(`${colLetter}1`).setValue("Notes");




  selectedSheet.getFreezePanes().freezeRows(1);

  const usedRangeStep5 = selectedSheet.getUsedRange();

  if (usedRangeStep5) {

    const totalRows = usedRangeStep5.getRowCount();

    const totalCols = usedRangeStep5.getColumnCount();




    if (totalRows > 1) {
```

```
    const bodyRange = selectedSheet.getRangeByIndexes(1, 0, totalRows - 1, totalCols);

    bodyRange.getFormat().getFill().setColor("#00B050");

  }

}



// STEP 6: VLOOKUP - Fill Notes column only where Column C has values

let vlookupUsedRange = selectedSheet.getUsedRange();

if (!vlookupUsedRange) return;



let vlookupValues = vlookupUsedRange.getValues();

let rowCount = vlookupValues.length;

let colCount = vlookupValues[0].length;
```

```
let vlookupHeaders = vlookupValues[0] as string[];

let notesColIndex = vlookupHeaders.indexOf("Notes");

let colCIndex = 2; // Column C



if (notesColIndex === -1) {

  console.log("Notes column not found.");

  return;

}



let formulas: (string | null)[][] = [];

formulas.push([null]); // Header stays empty
```

```
  for (let i = 1; i < rowCount; i++) {

    let lookupValue = vlookupValues[i][colCIndex];

    if (lookupValue !== "" && lookupValue !== null) {

      formulas.push([`=VLOOKUP(C${i + 1},'[Archived Outs.xlsx]SSC Dashboard
Details'!$C$2:$D$1000,2,FALSE)`]);

    } else {

      formulas.push([null]);

    }

  }



  let notesRange = selectedSheet.getRangeByIndexes(0, notesColIndex, rowCount, 1);

  notesRange.setFormulasLocal(formulas);
```

```
console.log("VLOOKUP inserted into Notes column.");


// STEP 7: Format, refresh, and sort Date column (Column D)

selectedSheet.getRange("D:D").setNumberFormatLocal("m/d/yyyy");


selectedSheet.getAutoFilter().apply(selectedSheet.getRange("1:1"));

selectedSheet.getAutoFilter().remove();


selectedSheet.getRange("D:D").setNumberFormatLocal("m/d/yyyy");


selectedSheet.getRange("D:D").copyFrom(selectedSheet.getRange("D:D"),
ExcelScript.RangeCopyType.values, false, false);
```

```
  selectedSheet.getAutoFilter().apply(selectedSheet.getRange("1:1"));

  selectedSheet.getAutoFilter().getRange().getSort().apply([{ key: 3, ascending: false }],
false, true);

}



function getColumnLetter(colIndex: number): string {

  let letter = "";

  while (colIndex > 0) {

    const mod = (colIndex - 1) % 26;

    letter = String.fromCharCode(65 + mod) + letter;

    colIndex = Math.floor((colIndex - mod) / 26);

  }
```

```
  return letter;

}
```